MODULE 25 · DOMAIN 3 - CLOUD NATIVE APPLICATION DELIVERY

Debugging applications

Probes, logs, events, exec, ephemeral debug containers, port-forward and top.

DOMAIN 3 of 4 EXAM WEIGHT 16% MODULE 25 of 35

admin@sthithapragna.com

The kubelet asks your app three questions: **have you started**, **are you alive**, and **are you ready**?

The three probes

PROBE	ASKS	IF IT FAILS
`startupProbe`	Has the app finished starting?	Keeps retrying; after its threshold, the container is restarted
`livenessProbe`	Is the app still alive, not stuck?	The container is restarted
`readinessProbe`	Can it take traffic right now?	Removed from Service endpoints. Not restarted

ON THE EXAM

Liveness restarts. Readiness removes from traffic. Startup holds the other two back until the app is up.

Liveness versus readiness

LIVENESS

Restart me

- For states only a restart fixes
- Deadlocks, a wedged event loop
- Checks only the process itself
- Failure: container restarted

READINESS

Do not send me traffic

- For temporary unavailability
- Warming up, overloaded, a dependency down
- May check what it needs to serve
- Failure: removed from endpoints

THE TIEBREAKER

If a restart would not fix it, it belongs in readiness, not liveness.

How probes check, and their defaults

MECHANISM OR SETTING	DETAIL
`httpGet`	Success on any 2xx or 3xx response
`tcpSocket`	Success if a TCP connection opens
`exec`	Success if the command exits with 0
`grpc`	Uses the standard gRPC health check
`periodSeconds / timeoutSeconds`	Defaults: every 10 seconds, 1-second timeout
`failureThreshold`	Default 3 failures in a row before acting

ON THE EXAM

Liveness with defaults acts after about 30 seconds of failure: three checks, ten seconds apart.

All three probes on the booking API

```
containers:
- name: api
  image: quayside/booking-api:2.4.1
  startupProbe:
    httpGet: {path: /healthz, port: 8080}
    failureThreshold: 30
    periodSeconds: 5
  livenessProbe:
    httpGet: {path: /healthz, port: 8080}
  readinessProbe:
    httpGet: {path: /ready, port: 8080}
```

6-7 Up to 30 x 5 = 150 seconds to start, before liveness begins.

9 Liveness: is the process healthy? Default timings once started.

11 Readiness: a separate endpoint that may check dependencies.

Probe mistakes that cause outages

- **Liveness checks a dependency.** The database blips, and every Pod restarts at once.
- **Slow start, no startup probe.** Liveness kills the app before it is ready: a crash loop.
- **No readiness probe.** Traffic reaches Pods before they can serve, on every rollout.
- **Heavy probe endpoints, checked too often,** add load and fail under pressure.

> ON THE EXAM
>
> A probe that checks the wrong thing is worse than no probe at all.

Look before you touch: logs and events first, then step inside only if you must.

kubectl logs, and the flags that matter

COMMAND	SHOWS
`kubectl logs pod`	The container's standard output and error
`-f`	Follow: keep streaming new lines
`-c api`	A specific container, in a multi-container Pod
`--previous (-p)`	The last instance, before it crashed
`--since=10m, --tail=100`	Only recent lines
`-l app=web`	Logs from every Pod with that label

ON THE EXAM

Crashed container: --previous. Multi-container Pod: -c. Both at once is common.

What logs can and cannot tell you

- **Only standard output and standard error.** Apps should log there, not to files.
- **Stored on the node and rotated.** Old lines disappear.
- **Gone with the Pod,** or the node. The previous instance is the furthest back you get.
- **For history,** ship logs to a central store: module 26.

> ON THE EXAM
>
> kubectl logs is for now. Central logging is for yesterday.

Events

- **kubectl describe** shows the events for one object, at the bottom.
- **kubectl get events -n quayside --sort-by=.lastTimestamp** shows a namespace's recent events.
- **Warning events** flag trouble: failed scheduling, failed pulls, failed probes, back-offs.
- **Kept for about an hour by default,** then deleted. Yesterday's events are gone.

> ON THE EXAM
>
> Events are the cluster's short-term memory. For anything older, you need monitoring and logs.

kubectl exec versus kubectl debug

KUBECTL EXEC

Use what is already there

- Runs a command in an existing container
- Needs the tool, even a shell, in the image
- Fails on distroless images
- Quick, when the tools exist

KUBECTL DEBUG

Bring your own tools

- Adds an ephemeral container to a running Pod
- Uses any image, such as busybox
- Works on distroless images
- Can see the app's processes with --target

THE TIEBREAKER

No shell in the image: kubectl debug with an ephemeral container.

Debugging a distroless container

```
$ kubectl exec -it ticket-validator-6c9d-x7k2p -- sh
error: exec: "sh": executable file not found in $PATH

$ kubectl debug -it ticket-validator-6c9d-x7k2p --image=busybox:1.37 --target=validator
/ # ps
PID   USER     COMMAND
    1 65532    /validator
```

2 Distroless: there is no shell to exec into.

4 An ephemeral busybox container, sharing the validator's process namespace.

7 The app itself, visible from the debug container.

Three ways to use kubectl debug

MODE	CREATES	USE WHEN
`kubectl debug pod --image=... --target=app`	An ephemeral container in the running Pod	Inspect a live Pod without changing it
`kubectl debug pod --copy-to=pod-debug ...`	A copy of the Pod, with changes such as a new image or command	The Pod crashes too fast to attach to
`kubectl debug node/node-3 -it --image=...`	A Pod on that node, using its namespaces; its filesystem at /host	Investigate the node itself

ON THE EXAM

Crash-looping too fast to attach? Debug a copy, with its command changed to a shell.

port-forward, top and cp

- **kubectl port-forward svc/booking-api 8080:80** reaches it at localhost:8080 on your laptop.
- **Nothing is exposed:** no Service type change, no Ingress. It tunnels through the API server.
- **kubectl top pods / nodes:** current CPU and memory, from metrics-server. Not history.
- **kubectl cp:** copies files to or from a container. Needs tar in the image.

> ON THE EXAM
>
> 'Reach it from my laptop without exposing it': kubectl port-forward.

Six debugging decisions. For each, ask: **which question is failing**, and **which tool answers it**?

> A new Java reporting service takes about 90 seconds to start. It has a liveness probe with default settings and no startup probe.
>
> It never becomes ready: every Pod is restarted after about 30 seconds, and ends in CrashLoopBackOff.

What is the best fix?

A. Remove the readiness probe from the container

B. Raise the CPU limit so it might start faster

C. Add a startup probe that allows enough time

D. Set the Pod's restartPolicy to Never

◆ PAUSE AND DECIDE

What is the best fix?

A Remove the readiness probe from the container
Readiness never restarts anything.

B Raise the CPU limit so it might start faster
May help a little; guarantees nothing.

C Add a startup probe that allows enough time
Holds liveness back until the app has started.

D Set the Pod's restartPolicy to Never
Stops restarts, but the app is still killed and never serves.

THE SIGNAL

Slow start killed by liveness: add a startup probe.

> During a two-minute database maintenance window, every booking API Pod was restarted, and the outage lasted fifteen minutes.
>
> The liveness probe's endpoint checks the database connection.

What change prevents this?

A Keep liveness to the process; check the DB in readiness

B Raise the liveness failureThreshold to 100 checks

C Remove every probe from the booking API

D Add more replicas of the booking API

◆ PAUSE AND DECIDE

What change prevents this?

A. Keep liveness to the process; check the DB in readiness
A database blip then only removes Pods from traffic, briefly.

B. Raise the liveness failureThreshold to 100 checks
Masks real deadlocks for many minutes.

C. Remove every probe from the booking API
Loses restarts and traffic control entirely.

D. Add more replicas of the booking API
More replicas would all restart at once too.

THE SIGNAL

Liveness checking a dependency causes cascading restarts. Move it to readiness.

> The distroless ticket validator is misbehaving in production. kubectl exec fails: there is no shell in the image.
>
> An engineer needs to inspect its processes and network from inside the running Pod, without restarting it.

What should they use?

A Rebuild the image with a shell, and redeploy it

B kubectl logs -f on the validator container

C kubectl port-forward to the validator Pod

D kubectl debug, an ephemeral container with --target

◆ PAUSE AND DECIDE

What should they use?

A Rebuild the image with a shell, and redeploy it
Replaces the misbehaving Pod, and the evidence with it.

B kubectl logs -f on the validator container
Shows output, but gives no view inside the container.

C kubectl port-forward to the validator Pod
Reaches its port; still no view of processes.

D kubectl debug, an ephemeral container with --target
Adds a tool container to the live Pod; no restart.

THE SIGNAL

No shell, must not restart, need to look inside: kubectl debug.

> The booking API Pod runs two containers: api and a log-shipper sidecar. The api container crashed and has just restarted.
>
> The engineer needs the api container's output from just before the crash.

Which command gives exactly that?

A kubectl logs booking-api-pod -f

B kubectl logs booking-api-pod -c api --previous

C kubectl describe pod booking-api-pod

D kubectl logs booking-api-pod --all-containers

◆ PAUSE AND DECIDE

Which command gives exactly that?

A kubectl logs booking-api-pod -f

Needs a container name, and shows only the current instance.

B kubectl logs booking-api-pod -c api --previous

The api container's previous, crashed instance.

C kubectl describe pod booking-api-pod

Shows events and exit codes, not the app's output.

D kubectl logs booking-api-pod --all-containers

Current instances only, mixed with the sidecar's lines.

THE SIGNAL

Crashed container in a multi-container Pod: -c plus --previous.

> A developer wants to test the staging booking API from their laptop for an hour.
>
> It must not be exposed outside the cluster, and nothing about the Service may change.

What should they use?

A. kubectl port-forward svc/booking-api 8080:80

B. Change the Service to type LoadBalancer

C. Add an Ingress rule for the booking API

D. Change the Service to type NodePort

◆ PAUSE AND DECIDE

What should they use?

A kubectl port-forward svc/booking-api 8080:80
A private tunnel for as long as the command runs.

B Change the Service to type LoadBalancer
Exposes it publicly and changes the Service.

C Add an Ingress rule for the booking API
Exposes it through the shared entrance.

D Change the Service to type NodePort
Opens it on every node, and changes the Service.

THE SIGNAL

'From my laptop, not exposed, nothing changed': port-forward.

> An engineer investigates a Pod that failed repeatedly yesterday afternoon. Today, kubectl get events shows nothing about it.
>
> The Pod still exists, and the engineer's permissions have not changed.

Why are the events missing?

A RBAC hides events older than one day

B Events are only recorded for Deployments

C Events expire after about an hour by default

D Events are cleared whenever kubectl restarts

◆ PAUSE AND DECIDE

Why are the events missing?

A RBAC hides events older than one day

Permissions did not change; RBAC has no age filter.

B Events are only recorded for Deployments

Pods have events too, as describe shows.

C Events expire after about an hour by default

They were deleted long ago. Use central logs and metrics.

D Events are cleared whenever kubectl restarts

kubectl holds no state; events live in the cluster.

THE SIGNAL

Yesterday's events are gone: they are short-term memory. History needs monitoring.

Six mixed decisions from **delivery, packaging, GitOps and debugging**, the whole domain in one pass.

> Quayside installs the same booking API chart twice, each with its own values file.
>
> One release is booking-staging, in the staging namespace. The other is booking-prod, in production.

In Helm terms, what are booking-staging and booking-prod?

A Two separate charts

B Two releases of one chart

C Two chart repositories

D Two revisions of one release

◆ PAUSE AND DECIDE

In Helm terms, what are booking-staging and booking-prod?

A Two separate charts

There is one chart, installed twice.

B Two releases of one chart

Each installed copy of a chart is a release.

C Two chart repositories

Repositories hold charts; nothing was added.

D Two revisions of one release

Revisions belong to one release over time.

THE SIGNAL

One chart, installed twice: two releases.

> The web front end has 9 stable Pods behind its Service. The team adds 1 canary Pod with the same labels, so the Service selects all 10.
>
> No gateway or mesh weights are configured.

Roughly what share of requests reaches the canary?

A. About half, since there are two versions

B. About 1%, the usual canary share

C. None, until the canary is promoted

D. About 10%, one Pod in ten

◆ PAUSE AND DECIDE

Roughly what share of requests reaches the canary?

A About half, since there are two versions
Services balance across Pods, not versions.

B About 1%, the usual canary share
Nothing sets 1%; the share follows Pod count.

C None, until the canary is promoted
The Service selects it, so it gets traffic now.

D About 10%, one Pod in ten
One of ten matching Pods: roughly a tenth.

THE SIGNAL

Canary by replica ratio: the share follows the Pod count.

> Quayside's second cluster uses Flux. The team wants to install a Helm chart declaratively, with its values kept in Git.
>
> Flux should then install and upgrade it automatically.

Which Flux object describes the chart and its values?

A. A Flux Kustomization

B. A HelmRelease

C. A GitRepository

D. An Argo CD Application

◆ PAUSE AND DECIDE

Which Flux object describes the chart and its values?

A. A Flux Kustomization
Applies manifests from a path; not a Helm chart.

B. A HelmRelease
The helm-controller's object: chart plus values.

C. A GitRepository
Where source files come from, not what to install.

D. An Argo CD Application
Argo CD's object, not Flux's.

THE SIGNAL

Flux plus a Helm chart: HelmRelease.

> Every rolling update of the web front end causes about twenty seconds of errors for some users.
>
> New Pods receive traffic the moment they start, but need about fifteen seconds to load their caches. There is no readiness probe.

What change fixes this?

A Increase maxSurge on the Deployment

B Switch the strategy to Recreate

C Add a liveness probe to the container

D Add a readiness probe to the container

◆ PAUSE AND DECIDE

What change fixes this?

A Increase maxSurge on the Deployment

More new Pods at once: more errors, not fewer.

B Switch the strategy to Recreate

Causes a full outage on every deploy instead.

C Add a liveness probe to the container

Restarts stuck containers; does not gate traffic.

D Add a readiness probe to the container

Traffic waits until each new Pod is ready.

THE SIGNAL

Errors during rollouts from Pods not yet warm: add readiness.

> Quayside runs Argo CD with six registered clusters, and adds more every quarter.
>
> The log agent must be deployed to every registered cluster, including ones added in future.

Which ApplicationSet generator fits?

A The cluster generator

B The list generator

C The Git directory generator

D The pull request generator

◆ PAUSE AND DECIDE

Which ApplicationSet generator fits?

A **The cluster generator**
One Application per registered cluster, new ones included.

B **The list generator**
A fixed list: new clusters need manual edits.

C **The Git directory generator**
One per folder in Git, not per cluster.

D **The pull request generator**
One per open pull request, for previews.

THE SIGNAL

'Every registered cluster, including future ones': cluster generator.

> Quayside's GitOps repository uses Kustomize: a shared base, and an overlay per environment that pins the image tag.
>
> Staging already runs booking API 2.5.0 successfully. Now production should move to 2.5.0.

What is the GitOps way to promote it?

A Run kubectl set image on the production Deployment

B Rebuild the image and retag it as latest

C A pull request changing newTag in the prod overlay

D Change the image tag in the shared base

◆ PAUSE AND DECIDE

What is the GitOps way to promote it?

A Run kubectl set image on the production Deployment

Drift: self-heal reverts it, and Git never records it.

B Rebuild the image and retag it as latest

Breaks build once, promote; latest moves.

C A pull request changing newTag in the prod overlay

A reviewed, recorded, revertable change.

D Change the image tag in the shared base

Changes every environment at once, not just prod.

THE SIGNAL

Promote by changing the production overlay in Git, through review.

Module 25 in five lines

- **Startup** holds the others back; **liveness** restarts; **readiness** removes from traffic.
- Liveness checks **only the process**; dependencies go in **readiness**. Defaults: 10s, 1s, 3.
- **logs -c name --previous** for a crashed container; logs vanish with the Pod.
- **kubectl debug** adds an ephemeral container: works on distroless, no restart.
- **port-forward** reaches it privately; events last **about an hour**.

That completes Application Delivery. Final domain: **Cloud Native Architecture**, starting with observability.